

Difference-Aware Contrastive Transformer for Physical Commonsense Question Answering

Group 73

Abstract

This project investigates difference-aware modelling for physical commonsense question answering on PIQA, where each goal has two near-identical candidate solutions. A Difference-Aware Contrastive Transformer (DACT), trained from scratch, marks the tokens in which the candidates differ, compares the candidates with cross-solution attention, pools around the differing tokens and adds a jointly trained lexical component. It was evaluated against five baselines and in nine ablations, and its attention was analysed. DACT achieved the highest validation accuracy among the models trained from scratch (61.3%); on the evaluation split, no statistically significant difference from a TF-IDF baseline was detected (61.9 vs. 61.2%, McNemar $p=0.62$). The model locates the differing words but lacks the knowledge to judge them.

1 Introduction

1.1 Background and Motivation

PIQA [1] is a question answering benchmark for physical commonsense: each item consists of an everyday goal and two candidate solutions, exactly one of which achieves the goal. Physical commonsense is relevant to assistants that give practical instructions and to robots that must choose feasible actions. The main difficulty of PIQA is that the two candidates are usually near-duplicates; in the training split used here, the median share of differing solution tokens is 15%. The correct answer therefore depends on a few substituted words whose plausibility requires world knowledge. This motivates an architecture that represents the differences explicitly instead of scoring each candidate independently.

1.2 Project Objectives

The objective of this project is to investigate whether a neural model trained from scratch, with-

out external data, can exploit the explicit differences between two candidate solutions to improve physical commonsense QA. The specific goals were: (i) to design and implement an attention-based architecture that represents the differences between the candidates in its input, its comparison of the candidates and its summary representation; (ii) to compare it with baselines ranging from a majority classifier to pretrained models; (iii) to measure the contribution of each component with controlled ablations; and (iv) to analyse whether the attention mechanism concentrates on the differing tokens and how this relates to the model's errors.

1.3 Related Work

Bisk et al. [1] report 77.1% accuracy for RoBERTa-large on PIQA against 94.9% for humans, and multi-format QA training improves pretrained models further [2]; these figures are cited for context only. The main model in this project builds on the Transformer encoder [3]. For multiple-choice reading comprehension, the Option Comparison Network [4] and DCMN+ [5] compare candidate answers with each other instead of scoring them independently. For natural language inference, ESIM [6] and the decomposable attention model [7] align two texts token by token and build comparison features from the alignment. Work on annotation artefacts shows that shallow lexical cues can solve many items without reasoning [8, 9]; this motivates both the lexical baseline and the lexical component of the model.

2 Methodology

DACT is a Transformer-based model trained from scratch on the PIQA training data only (Figure 1). The Transformer encoder, additive attention, BPE and the ESIM comparison operations are taken from prior work; the project-specific design con-

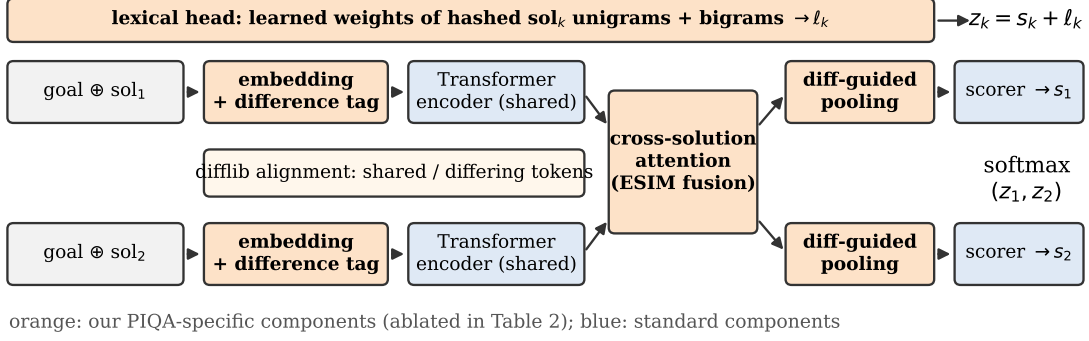


Figure 1: Architecture of DACT. Both candidates are processed by the same weights. Orange boxes are the project-specific components evaluated in the ablation study; blue boxes are standard components. Because the weights are shared, swapping two already-encoded candidates swaps their scores; the `difflib` alignment that produces the tags depends on the option order (Section 4.6).

sists of the difference-tag embeddings, the adaptation of cross-solution comparison to two competing PIQA solutions, the difference-guided pooling and the joint training of a lexical scoring component with the neural model.

2.1 Task Formulation

Each item consists of a goal g and two candidate solutions s_1 and s_2 . The model computes a score z_k for each candidate $k \in \{1, 2\}$ and predicts $\hat{y} = \arg \max_k z_k$.

2.2 Input Representation and Difference Tags

Each candidate is encoded as $x^{(k)} = [\text{CLS}] g [\text{SEP}] s_k [\text{SEP}]$ using a BPE vocabulary [10] learned on the training split. The token sequences of s_1 and s_2 are aligned with Python’s `difflib`, and every solution token receives a tag $t_i \in \{\text{shared}, \text{different}\}$; goal and special tokens receive the tag *other*. The input embedding of position i is

$$e_i = E_{tok}[x_i] + E_{pos}[i] + E_{seg}[\sigma_i] + E_{tag}[t_i], \quad (1)$$

where E_{tok} , E_{pos} , E_{seg} and E_{tag} are learned token, position, segment and tag embeddings and σ_i indicates whether position i belongs to the goal or the solution. The tag embedding informs the encoder from the first layer where the candidates disagree.

2.3 Shared Transformer Encoder

A pre-LayerNorm Transformer encoder [3, 11] maps each candidate sequence to hidden states $H^{(k)} \in \mathbb{R}^{L \times d}$. Both candidates share the encoder,

so differences between their representations arise from the inputs, not from different parameters.

2.4 Cross-Solution Attention

Plausibility in PIQA is relative: a solution is correct because it is better than its rival. Each token of candidate k therefore attends to the solution tokens of the other candidate $\bar{k}$, plus the CLS token of $\bar{k}$, which serves as a target when a token has no counterpart:

$$c_i = \text{MHA}(h_i^{(k)}, H^{(\bar{k})}, H^{(\bar{k})}), \quad (2)$$

where MHA is multi-head attention. The hidden states are then updated with the comparison features of ESIM [6]: $\tilde{h}_i = \text{LN}(h_i + \text{FFN}([h_i; c_i; h_i - c_i; h_i \odot c_i]))$. ESIM applies these features to premise–hypothesis pairs; this project applies them to two competing answers, so that each candidate is represented by what it contains instead of its rival.

2.5 Difference-Guided Attention Pooling

The solution tokens are summarised with additive attention [12] extended by a learned bias per tag:

$$\alpha_i \propto \exp(v^\top \tanh(W\tilde{h}_i) + b_{t_i}), \quad p = \sum_i \alpha_i \tilde{h}_i, \quad (3)$$

where v and W are learned and $b_{\text{different}}$ is initialised to 1 and the other biases to 0. The bias directs the summary towards the changed tokens at the start of training; Section 4.3 tests whether the trained model relies on it. A scorer MLP maps $[p; \tilde{h}_{\text{CLS}}]$ to a neural score s_k .

2.6 Lexical Scoring Component

Following the wide-and-deep principle [13], a sparse linear component scores the words of each solution directly. Every BPE unigram and bigram of s_k is hashed into one of 2^{18} buckets with a learned weight w_j , initialised to 0:

$$\ell_k = \sum_{j \in \text{ngrms}(s_k)} w_{h(j)}, \quad z_k = s_k + \ell_k, \quad (4)$$

where h is the hash function. Because the final prediction depends only on $z_1 - z_2$, n-grams shared by both candidates cancel, and the component learns which substituted words indicate a plausible solution, the signal that a TF-IDF classifier uses. It is trained jointly with the Transformer, with its own learning rate.

2.7 Prediction and Training Objective

The two scores compete in a softmax, $P(k) = \text{softmax}(z_1, z_2)_k$, which is trained with cross-entropy and label smoothing 0.1. This pairwise objective matches PIQA’s forced-choice format, in which exactly one of the two candidates is correct. Before this objective, the selected configuration runs a 10-epoch masked-language-model (MLM) warm-up on the text of the training split [14].

3 Experimental Setup

3.1 Dataset and Data Split

PIQA provides 16,113 labelled training items and 1,838 labelled development items; the test labels of the official benchmark are not public. The 1,838 development items serve as this project’s final evaluation split and are called the evaluation split below. A stratified 10% of the training file (seed 42) was held out for validation, giving 14,501 / 1,612 / 1,838 items for training, validation and evaluation, all label-balanced (50.0 / 50.0 / 50.5% label 1). An audit of the data found six duplicate rows within the training file and one evaluation item (index 1545) that appears twice in the training split. The split was kept unchanged and the overlap is disclosed: re-scoring the saved evaluation predictions without that item changes accuracy by about 0.02 points (DACT 61.86 $\rightarrow$ 61.84%, TF-IDF 61.21 $\rightarrow$ 61.19%).

3.2 Data Preprocessing

Whitespace is collapsed, and the BPE tokenizer applies NFKC normalisation and lower-casing. Goals are truncated to 40 and solutions to 96 BPE

tokens. For longer solutions (1.6% of evaluation solutions), the 96-token window is centred on the differing span instead of keeping the prefix.

3.3 Training Configuration

The final model has $d=128$, two layers, four heads and a feed-forward width of 512, with 1.93 M parameters, 262 k of them in the lexical component. It is trained with AdamW [15] ($\beta_2=0.98$, weight decay 0.05, learning rate 3×10^{-4} ; lexical component 10^{-2}), 6% linear warm-up followed by cosine decay, batches of 128 items, gradient clipping at norm 1.0 and dropout 0.2, for at most 20 epochs with early stopping on validation accuracy (patience 6). The MLM warm-up runs for 10 epochs at learning rate 5×10^{-4} with 15% of the tokens masked. All models trained from scratch use seeds 42–44 and fp16 mixed precision on a free Colab T4. The configuration was selected on validation with one seed from five candidates: base ($d=256$, 4 layers) 61.8%, small ($d=128$, 2 layers, 4 heads) 61.4%, base with dropout 0.3 60.8%, and base/small with the MLM warm-up 60.8 / **61.8%**. The selected model is small with the warm-up, 0.06 points ahead of base. The three best candidates lie within 0.43 points, less than one seed std of the final model (0.5), so the selection only rules out clearly worse settings (Section 4.5). The lexical learning rate was selected in a separate validation pilot.

3.4 Baseline Models

All baseline results were produced by the group’s own experiments. **B0** predicts the majority class and gives the chance level. **B1** is a logistic regression on TF-IDF features of the 1–2-grams of $s_2 - s_1$, with C tuned on validation; it measures how far shallow lexical cues go [8]. **B2** is a two-layer BiLSTM with additive attention and the same tokenizer (5.1 M parameters), a recurrent model trained from scratch with a similar budget. The following two are pretrained models used as baselines only, not as the main model. **B3** fine-tunes roberta-base [16] (125 M parameters) with a multiple-choice head (4 epochs, learning rate 2×10^{-5} , 3 seeds, best epoch on validation); a seed below 52% validation accuracy after epoch 1 is restarted with a new seed (a rule declared before the run). Because this fine-tuning proved unstable, B3 was also re-run with unchanged settings on validation only, in a pre-registered stability test. **B4** applies Qwen2.5-1.5B [17] zero-shot: the prompt

“Goal: g / Solution:” is followed by each solution, and the solution with the higher length-normalised log-likelihood is chosen, which measures the knowledge of a pretrained LLM without any training.

3.5 Evaluation Metrics

Accuracy is PIQA’s official metric. Because the labels are balanced and each item has exactly one correct candidate, accuracy is unbiased and has a chance level of 50%. Models trained with seeds 42–44 are reported as mean $\pm$ standard deviation. For the seed with the best validation accuracy, a 95% bootstrap confidence interval (2,000 resamples [18]) and an exact McNemar test [19] against DACT are computed on the evaluation split.

3.6 Experimental Protocol

Validation accuracy is the evidence used for every design and selection decision. The evaluation split was, however, evaluated in several development runs, and the lexical component was introduced after earlier evaluation results had been seen; its configuration was then chosen on validation only. The evaluation results are therefore useful for comparison but are not an untouched estimate of generalisation. In the final run, evaluation labels are read only in the last step of the notebook, and every read is logged. The submitted notebook contains the complete final run (commit b880108; 57 cells, no errors, logs of all 33 training runs) and a cell that checks the tables against the result files.

4 Results and Analysis

4.1 Overall Performance

Table 1 compares DACT with the baselines. DACT has the highest accuracy among the models trained from scratch on both splits: +2.0 / +3.1 points over the BiLSTM (McNemar $p=0.09$ on test) and +2.0 / +0.7 over TF-IDF. The 2-point validation lead over TF-IDF was also observed in an earlier independent A100 run, whereas on the evaluation split no statistically significant difference was detected ($p=0.62$; $p=0.88$ in the A100 run). The two models make substantially different errors (Section 4.2), so the result reflects two different sources of evidence of similar strength rather than one model reproducing the other. The pretrained baselines perform better: the zero-shot LLM (about 800 times more parameters) is 13.2 points above DACT on test, and RoBERTa (65

Model	Val	Test
<i>trained from scratch</i>		
B0 majority	50.0	50.5
B1 TF-IDF + LR	59.2	61.2
B2 BiLSTM + attention	59.3 $\pm$ 0.7	58.7 $\pm$ 1.1
DACT (main model)	61.3 $\pm$ 0.5	61.9 $\pm$ 0.3
<i>pretrained baselines</i>		
B3 RoBERTa-base, fine-tuned [†]	59.2 $\pm$ 5.8	57.3 $\pm$ 6.9
validation re-run [‡]	69.0 $\pm$ 0.6	–
B4 Qwen2.5-1.5B, zero-shot	78.2	75.1

Table 1: Accuracy (%) on the validation and evaluation (Test) splits; mean $\pm$ std over 3 seeds; B0, B1 and B4 are deterministic single runs. DACT test 95% CI: 59.6–64.0. [†]Final run: two of three seeds did not leave chance-level training loss ($\ln 2$); the working seed reaches 65.3% (test CI 62.4–66.6). [‡]Same code, settings and GPU type; all three seeds trained (69.6 / 68.5 / 68.8); validation only. An A100 run gave 68.0 / 66.7.

Ablation (vs. full DACT)	Δ Val	Δ Test
– lexical head	–2.3	–0.3
pointwise (independent BCE) objective	–1.7	–1.4
– difference tags	–0.2	–1.3
mean instead of attention pooling	–0.2	+0.2
– MLM warm-up	+0.1	–0.3
– tag bias in pooling	+0.1	–0.6
tag bias initialised at 0	+0.2	–0.7
– cross-solution attention	+0.6	–0.6
vanilla (four components removed)	–3.7	–4.3

Table 2: Ablation results: change in accuracy (percentage points) relative to the full model, 3 seeds each, one change at a time on the selected configuration; differences of the unrounded 3-seed means, rounded to 0.1 (seed std $\approx$ 0.1–1.1). The vanilla model removes the difference tags, cross-solution attention, attention pooling and the lexical component.

times more parameters), when its fine-tuning succeeds, is about 8 points above on validation (69.0 vs. 61.3%). Because model size, tokenisation, architecture and pretraining all differ, this gap cannot be attributed to world knowledge alone. RoBERTa’s fine-tuning trained one of three seeds in the final run and all three in the re-run with identical settings, a known instability [20].

4.2 Ablation Study

Each ablation tests one design hypothesis (Table 2); seeds, data and training budget are fixed. Removing the difference-aware components together (the vanilla model) reduced accuracy by 3.7 / 4.3 points, the largest drop and the only significant one on test ($p=0.007$); in the A100 run the same comparison cost 2.5 / 2.4. This indicates

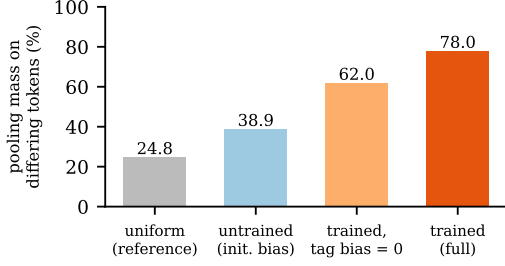


Figure 2: Share of pooling attention on differing tokens (evaluation split) for uniform attention, an untrained model, the trained model with the tag bias set to 0 and the full trained model.

that the combined design provides information beyond a plain Transformer. The lexical component mainly helps on validation (removing it costs 2.3 / 0.3 points), and the forced-choice objective helps on both splits (independent binary scoring: 1.7 / 1.4). The difference tags help on test only (0.2 / 1.3). The remaining components show no consistent effect: cross-solution attention (removing it gives +0.6 / -0.6), attention pooling (+0.2 on test with mean pooling), the tag bias and its initial value, and the MLM warm-up (+0.1 / -0.3) change accuracy by less than a point in opposite directions on the two splits. A likely explanation is that these components carry overlapping information about where the candidates differ, so each can be compensated by the others. Differences below one seed std are not conclusive, and with nine comparisons individual p-values should not be over-interpreted [21].

DACT and TF-IDF agree on only 78.2% of test items; DACT alone solves 11.2% (206 items) and TF-IDF alone 10.6%, for an oracle accuracy of 72.4%. The items only DACT solves include minimal pairs with a one-word difference inside long shared text (*release* vs. *do not release* the ignition key; the typo *sparrow bottle* vs. *spray bottle*), where the TF-IDF difference vector $s_2 - s_1$ is close to zero and the alignment-based tags are most informative. Against RoBERTa, the oracle rises to 82.4%.

4.3 Attention Analysis

The trained model puts 78.0% of its pooling attention on differing tokens, which make up 24.8% of solution tokens (Figure 2). Because the tag bias starts at 1, two controls separate learned focus from the built-in prior: an untrained model reaches only 38.9%, while the trained model with

the bias set to 0 still reaches 62.0%, and the bias itself barely changed in training (1.0 $\rightarrow$ 1.03). The focus is therefore mainly learned by the pooling scorer, which is consistent with the small effect of initialising the bias at 0. On validation, the normalised pooling entropy is 0.59 (1 = uniform), for correct and wrong predictions alike ($p=0.98$), so the pooling is selective in both cases. A linear probe separating differing from shared tokens reaches 62.0% balanced accuracy on the vanilla Transformer’s embeddings and only 63.0% after its encoder, whereas in DACT the tag remains perfectly decodable (100%) in every layer: without tags, the encoder does not learn from 14.5 k items where the candidates differ. Incorrect predictions put more attention on the differing tokens than correct ones (79.0 vs. 77.4%, $p=0.03$). Detecting the relevant difference is therefore not sufficient to judge physical plausibility. The attention distribution shows where the model places weight during aggregation, but it does not by itself establish the reason for a prediction [22, 23].

4.4 Qualitative Analysis

Figure 3 shows two evaluation items. For *adding scents to lotion bars* (top), the incorrect option adds the oils “to the water in the bottom of the double boiler”. The pooling peaks on *bottom*, the part that makes the option wrong, yet the model prefers this option with $p = 0.96$: it locates the decisive span but has no knowledge that oil in the water bath never reaches the lotion. In *a small but cheap home* (bottom), the candidates differ in a one-letter typo, a *shipping* vs. a *shopping* container, and in *living space* vs. *relaxing environment*. Almost half of the pooling weight is on *shipping*, but the scores tie. Correct predictions rest on similar word-level evidence: for *a simple camping lantern*, the correct option (a milk jug on top of an *LED light, translucent plastic*) wins with $p = 0.99$ against *fairy lights* in a filled jug, with pooling spread over the many differing words. The most confident error, *to make the most of an exotic trip* ($p = 0.98$ for a strict itinerary over flexible plans), illustrates an over-confidence of the lexical component: the two options share almost no words, and because ℓ_k sums over n-grams, long differing spans receive extreme scores. Single examples illustrate, but do not establish, general behaviour.

Goal: Adding scents to lotion bars

Option 1, $p = 0.96$ grate the wax and melt, in a double boiler, add the oils to the water in the bottom of the double boiler.

Option 2 (gold), $p = 0.04$ once the lotion and wax have melted, and reached the trace stage, add the scented oils and stir to combine.

Goal: How to make a small but cheap home easily?

Option 1 (gold), $p = 0.50$ purchase a shipping container online and have it installed on a cheap rental lot, then convert the container into a living space

Option 2, $p = 0.50$ purchase a shopping container online and have it installed on a cheap rental lot, then convert the container into a relaxing environment

Figure 3: Pooling attention over each candidate solution (darker = higher weight; **bold** = tokens that differ between the candidates). Top: *adding scents to lotion bars*, an incorrect prediction ($p = 0.96$ for option 1; option 2 is correct). Bottom: *a small but cheap home*, the most uncertain item ($p = 0.50$; option 1 is correct).

4.5 Experimental Stability

All development runs are logged in the repository. Difference-aware truncation removed inputs that had become identical after prefix truncation (in an early run, one item differed only at BPE positions 97–98). Giving the lexical component its own learning rate was necessary: with the shared rate, it barely moved from zero before early stopping. A goal-matching block (165 k parameters) was pre-registered, with its acceptance rule (validation gain ≥ 1.0 point and > 2 pooled seed std) committed before training; it lost 0.3 points on validation and was rejected without a test run. On the free T4, emulated bf16 slowed training from 108 to 377 ms per step until bf16 was restricted to GPUs that support it natively. Results were sensitive to seeds and sessions: between the A100 run and the final run, single ablations and the from-scratch baselines moved by up to 1.6 points, the one-seed grid chose a different configuration, and RoBERTa’s fine-tuning succeeded in one, two or three of three seeds depending on the session. The conclusions of this report therefore rest on effects larger than this spread, such as the vanilla ablation and the gap to pretrained models.

4.6 Limitations

Improvements of about one point are within seed and sampling noise, and each grid candidate was trained with one seed. The evaluation split was exposed during development (Section 3.6). One evaluation item overlaps with the training data, and the difference tags depend on the option order (102 evaluation items change tags when the options are swapped; an optional canonical align-

ment removes this). The model overfits quickly, with 85–92% training accuracy against about 61% validation accuracy at the selected epoch, so early stopping does most of the regularisation. Ablations that remove modules also change the parameter count. The comparison with pretrained models confounds knowledge with model size and pre-training, and attention weights are not causal explanations.

5 Conclusion

This project developed and evaluated a Difference-Aware Contrastive Transformer for PIQA, trained from scratch. Making the difference between two near-identical solutions explicit and adding lexical evidence gave the highest validation accuracy among the from-scratch models in two runs; on the evaluation split, no significant difference from TF-IDF was detected. The combined design gave the clearest improvement (3.7 / 4.3 points over a vanilla Transformer), while single components had no consistent effect. The model learns to focus on the differing words but still misjudges them; models with 65–800 times more parameters and pretraining reach 69–75%. Next steps are a length-normalised lexical score, a cleaner evaluation protocol and difference-aware mechanisms on pretrained representations.

Team Contributions

The work was divided among the three members; each member implemented and tested their own part of the code and wrote the report sections that describe it.

Bayron: original DACT pipeline (data processing, model, training and evaluation code, baselines B0–B4), the original A100 run, the final A100 runs of both versions and their comparison, and integration of the final notebook and report.

Ali: model revision (difference-aware truncation, lexical component and its validation pilot), attention controls and diagnostics, three-seed RoBERTa baseline, the free-T4 runs, and the first draft of the report.

Salah: evidence audit of the original run (training/evaluation overlap, order-dependent alignment, evaluation exposure), robustness and provenance code (validation, determinism, prediction export), and the local replication of the original model.

References

- [1] Y. Bisk, R. Zellers, R. Le Bras, J. Gao, and Y. Choi, “PIQA: Reasoning about physical commonsense in natural language,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 05, 2020, pp. 7432–7439.
- [2] D. Khashabi, S. Min, T. Khot, A. Sabharwal, O. Tafjord, P. Clark, and H. Hajishirzi, “UnifiedQA: Crossing format boundaries with a single QA system,” in *Findings Assoc. Comput. Linguistics: EMNLP 2020*, 2020, pp. 1896–1907.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [4] Q. Ran, P. Li, W. Hu, and J. Zhou, “Option comparison network for multiple-choice reading comprehension,” *arXiv preprint arXiv:1903.03033*, 2019.
- [5] S. Zhang, H. Zhao, Y. Wu, Z. Zhang, X. Zhou, and X. Zhou, “DCMN+: Dual co-matching network for multi-choice reading comprehension,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 05, 2020, pp. 9563–9570.
- [6] Q. Chen, X. Zhu, Z.-H. Ling, S. Wei, H. Jiang, and D. Inkpen, “Enhanced LSTM for natural language inference,” in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, 2017, pp. 1657–1668.
- [7] A. Parikh, O. Täckström, D. Das, and J. Uszkoreit, “A decomposable attention model for natural language inference,” in *Proc. 2016 Conf. Empirical Methods Natural Lang. Process.*, 2016, pp. 2249–2255.
- [8] S. Gururangan, S. Swayamdipta, O. Levy, R. Schwartz, S. Bowman, and N. A. Smith, “Annotation artifacts in natural language inference data,” in *Proc. 2018 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (Vol. 2: Short Papers)*, 2018, pp. 107–112.
- [9] A. Poliak, J. Naradowsky, A. Haldar, R. Rudinger, and B. Van Durme, “Hypothesis only baselines in natural language inference,” in *Proc. 7th Joint Conf. Lexical Comput. Semantics*, 2018, pp. 180–191.
- [10] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, 2016, pp. 1715–1725.
- [11] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, “On layer normalization in the transformer architecture,” in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 10 524–10 533.
- [12] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *Proc. Int. Conf. Learn. Represent.*, 2015.
- [13] H.-T. Cheng, L. Koc, J. Harmsen, T. Shaked, T. Chandra, H. Aradhye, G. Anderson, G. Corrado, W. Chai, M. Ispir, R. Anil, Z. Haque, L. Hong, V. Jain, X. Liu, and H. Shah, “Wide & deep learning for recommender systems,” in *Proc. 1st Workshop Deep Learn. Recommender Syst.*, 2016, pp. 7–10.
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2019, pp. 4171–4186.
- [15] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learn. Represent.*, 2019.
- [16] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [17] Qwen Team, “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [18] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.
- [19] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [20] J. Dodge, G. Ilharco, R. Schwartz, A. Farhadi, H. Hajishirzi, and N. A. Smith, “Fine-tuning pre-trained language models: Weight initializations, data orders, and early stopping,” *arXiv preprint arXiv:2002.06305*, 2020.
- [21] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, “The hitchhiker’s guide to testing statistical significance in natural language processing,” in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, 2018, pp. 1383–1392.
- [22] S. Jain and B. C. Wallace, “Attention is not explanation,” in *Proc. 2019 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2019, pp. 3543–3556.
- [23] S. Wiegrefe and Y. Pinter, “Attention is not not explanation,” in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process.*, 2019, pp. 11–20.